

Fast Power Algorithm in C

Topic: Exponentiation by Squaring (Divide and Conquer)

Important Note

The supplied program is **not Prim's Algorithm**. It calculates x raised to the power n using a recursive divide-and-conquer technique called **Exponentiation by Squaring**.

C Program

```
#include <stdio.h>

long long power(int x, int n)
{
    if (n == 0)
        return 1;

    long long half = power(x, n / 2);

    if (n % 2 == 0)
        return half * half;
    else
        return x * half * half;
}

int main()
{
    int x, n;

    printf("Enter the number: ");
    scanf("%d", &x);

    printf("Enter the power: ");
    scanf("%d", &n);

    printf("%d^%d = %lld", x, n, power(x, n));

    return 0;
}
```

How the Algorithm Works

- **1. Base case:** If $n = 0$, the function returns 1 because $x^0 = 1$.
- **2. Divide:** It recursively calculates $\text{power}(x, n/2)$.
- **3. Store half:** The result is stored in the variable **half**.
- **4. Even power:** If n is even, $x^n = (x^{n/2}) \times (x^{n/2}) = \text{half} \times \text{half}$.
- **5. Odd power:** If n is odd, $x^n = x \times (x^{n/2}) \times (x^{n/2})$.
- **6. Return:** The calculated result is returned to the calling function.

Example

For 2^5 : $\text{power}(2,5) \rightarrow \text{power}(2,2) \rightarrow \text{power}(2,1) \rightarrow \text{power}(2,0)$. Then the calls return upward: $1 \rightarrow 2 \rightarrow 4 \rightarrow 32$. Therefore, $2^5 = 32$.

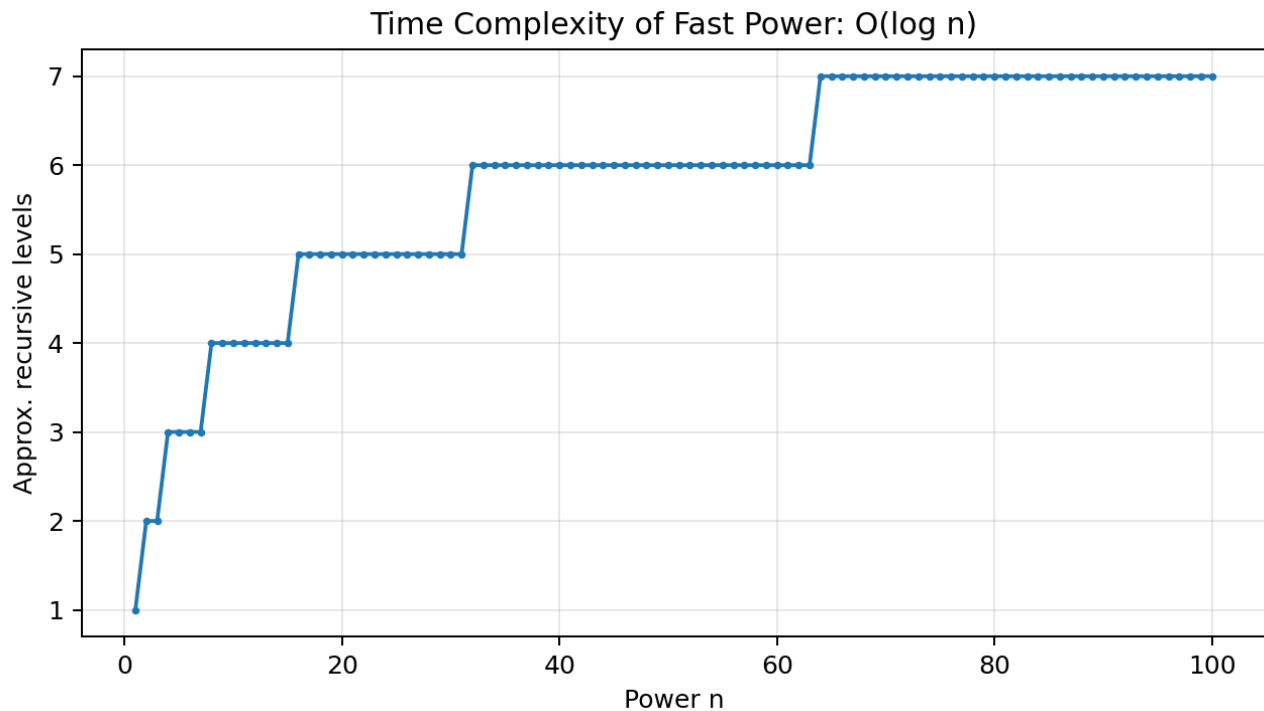
Time Complexity

Case	Complexity	Reason
Best / Average / Worst	$O(\log n)$	n is divided by 2 at every recursive call
Space Complexity	$O(\log n)$	Recursive call stack has about $\log_2(n)$ levels

Why $O(\log n)$?

The recurrence is approximately $T(n) = T(n/2) + O(1)$. Since n becomes $n/2$, $n/4$, $n/8$, ... until it reaches 1, the number of levels is about $\log_2(n)$. Therefore, the time complexity is $O(\log n)$, which is much faster than the simple $O(n)$ recursive approach.

Time Complexity Graph



The graph shows that the number of recursive levels grows logarithmically as n increases. The curve rises slowly compared with a linear $O(n)$ algorithm.

Key Points for Exam

- Algorithm name: **Exponentiation by Squaring / Fast Power**.
- Design technique: **Divide and Conquer**.
- Time complexity: **$O(\log n)$** .
- Auxiliary space complexity: **$O(\log n)$** because of recursion.
- Main idea: divide the exponent by 2 and reuse the computed half.